01. Spark Introduction

Created by	Anjali mane
Last edited	@March 29, 2026 5:12 PM
Tags	

Introduction to Apache Spark

- Apache Spark is an open-source, distributed data processing engine designed for fast. scalable processing of large datasets. It provides in-memory computing capabilities that make it significantly faster than traditional disk-based processing frameworks, particularly for iterative alogorithms and real-time analytics.
- Apache Spark is a unified analytics engine that provides parallel and distributed processing of big data workloads. It offers a comprehensive ecosystem of libraries for batch processing, SQL queries, streaming data, machine learning, and graph computation, all integrated into a single framework.

Why was it created

> Spark was created to address the limitations of Hadoop MapReduce, particularly its inefficiency for iterative processing required by machine learning alogorithms and interactive data analysis. The goal was to build a framework optimized for fast iterative processing while retaining the scalability and fault tolerance of Hadoop MapReduce.

How does it work?

> Spark processes data by keeping it in memory(RAM) rather than writing to disk after each operation. It uses a Directed Acyclic Graph (DAG) execution engine that optimizes task scheduling and reduces latency by eliminating unnecessary disk I/O operations. Spark Core provides essential services like task scheduling, memory management, fault recovery, and job execution, while higher-level components handle specific workloads.

Features of Apache Spark

1. **In-memory computation:** Data is processed in RAM instead of reading/writing to disk, making Spark significantly faster than traditional frameworks like Hadoop.
2. **Distributed processing using parallelize:** Data is split into partitions and processed simultaneously across multiple nodes/cores, reducing overall processing time.
3. **Can be used with many cluster managers (Spark, Yarn, Mesos e.t.c):** Spark is flexible - it can run on its own standalone cluster, or integrate with existing cluster managers like YARN(Hadoop) or Mesos.
4. **Fault-tolerant:** If a node fails during processing, Spark automatically recovers the lost data using RDD lineage (it remembers the steps to recreate the data).
5. **Immutable:** Once an RDD or DataFrame is created, it cannot be modified - every transformation creates a new dataset, ensuring data consistency and safe parallel processing.
6. **Lazy evaluation:** Transformations are not executed immediately - Spark builds an execution plan first and only runs it when and Action (like collect() or count()) is called, optimizing performance.
7. **Cache & persistence:** Frequently used DataFrames can be stored in memory using cache() or persist() so Spark doesn't recompute them every time they are needed.
8. **In-built optimization when using DataFrames:** DataFrames use the Catalyst Optimizer which automatically rewrites and optimizes your query execution plan for the best performance.
9. **Supports ANSI SQL:** You can run standard SQL queries directly on dataFrames using spark.sql(), making it easy for SQL-familiar engineers to work with Spark.

Issues with Traditional Systems

> Traditional data processing systems face several critical limitations when handling big data:

1. **Scalability bottlenecks:** Traditional systems scale vertically (adding more powerful hardware to a single machine) rather than horizontally, quickly hitting hardware limits as data grows from gigabytes to terabytes
2. **Lack of real- time processing:** Designed primarily for batch processing, traditional systems cannot support real-time analytics needed for modern use cases like fraud detection and personalized experiences.
3. **Performance issues:** Centralized relational databases slow down significantly as data volume increases, with each operation taking longer due to sequential

processing.

4. **Inflexibility with data types:** Traditional systems handle only structured data in rigid schemas, missing approximately by 80% of available data that exists in unstructured or semi-structured formats
5. **High costs:** Scaling traditional systems require expensive hardware upgrades and enterprise database licenses, with costs escalating exponentially.

Hadoop MapReduce

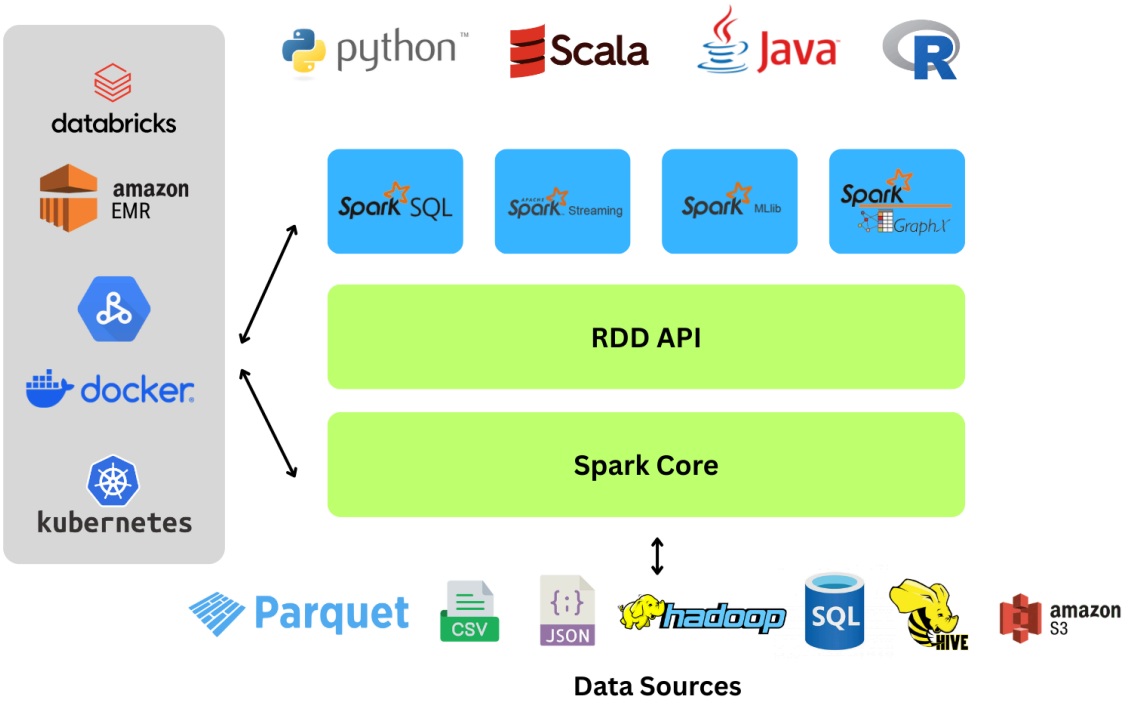
> Hadoop MapReduce is a programming model for processing large datasets in a distributed computing environment. It divides tasks into two independent phases: Map phase (which filters and sorts data) and the Reduce phase (which performs summary operations). While MapReduce introduced horizontal scalability and fault tolerance, it processes data by writing intermediate results to disk after each operation, creating significant I/O overhead.

MapReduce vs Spark Comparison

Aspect	Hadoop MapReduce	Apache Spark
Processing Speed	Slower due to disk-based storage between operations	100x faster in-memory, 10x faster on disk
Data Storage	Stores intermediate results on local disk	Caches data in RAM across distributed workers
Execution Model	Rigid sequential map and reduce phases	DAG-based execution with optimized task scheduling
Fault Tolerance	Replicates data to disk; re-reads from HDFS on Failure	Uses lineage tracking; recomputes lost partitions from source data
Processing Type	Primarily batch processing	Batch, real-time streaming, interactive queries
Ease of Use	Complex, requires more code	User-friendly APIs in multiple languages
Iterative Processing	Inefficient for machine learning algorithms	Optimized for iterative algorithms
Latency	High I/O overhead creates bottlenecks	Low latency with in-memory computation
Cost	Lower hardware requirements	Requires more memory resources

Spark Ecosystem

The Image below illustrates Apache Spark as a layered, unified stack where higher-level libraries rely on the core engine, allowing developers to use different languages and data sources seamlessly.



1. **Language APIs (Top Layer)** At the top, the diagram shows support for four major programming languages:
   a. Python
   b. Scala
   c. Java
   d. R

   This flexibility allows Data Engineers and Data Scientists to interact with Spark using the language they are most comfortable with, without needing to learn low-level cluster programming.
2. **Domain-Specific Libraries (Blue layer)** The four blue boxes represent Spark's built-in libraries that handle specific data workload types:
   a. Spark SQL: For processing structured data using SQL queries and DataFrames.

   b. Spark Streaming: For ingesting and processing real-time data streams.
   c. Spark MLib: A library of machine learning algorithms for tasks like classification and clustering.
   d. Spark GraphX: For graph computation and network analysis.
3. **The Core Engine (Green Layers)** The foundation of the stack is depicted by the two green bars: Spark Core and the RDD API.
   a. Spark Core: This is the underlying execution engine responsible for memory management, task scheduling, and fault recovery.
   b. RDD API: Sitting directly on top of the core, this exposes the Resilient Distributed Dataset abstraction, which is the fundamental data structure Spark uses to process data in parallel across the cluster.
4. **Deployment Environment** The “Environment” section on the left illustrates that Spark is decoupled from the resource manager. It can run in various environments.
   a. Containerized: Using Docker or Kubernetes.
   b. Cloud: On services like Amazon EC2, Databricks, Google DataProc.
   c. Cluster Managers: natively on Mesos or OpenStack.
5. **Data Sources (Bottom layer)** The bottom section shows Spark’s ability to connect to a wide variety of data storage systems. Instead of locking you into a single storage format, it can ingest data from:
   a. Cloud Storage: ADLS, GCS, AWS S3
   b. Hadoop Ecosystem: HDFS(Hadoop), Hive and HBase.
   c. NoSQL Databases: Cassandra, Elastisearch, MongoDB, Redis, and DynamoDB.
   d. Relational Databases: MySQL, PostgreSQL, Oracle, SQL Server, and Redshift.
   e. Flat Files: CSV, JSON, Parquet, Avro, and ORC formats.
   f. Streaming Sources: Apache Kafka, kinesis, and event hubs for real-time data ingestion.
   g. Cloud Data Warehouses: Snowflake, BigQuery, and Databricks Delta Lake.